

Evaluating and Stress-Testing Smart Contract Security Analyzers

Tamer Abdelaziz, Ph.D.

tamer.m@nyu.edu
[tamer-abdelaziz.github.io](https://github.com/tamer-abdelaziz)

Abstract

The effectiveness of smart contract security analyzers remains difficult to assess comprehensively, particularly under adversarial conditions involving obfuscation and composition. This project constructs an adversarial benchmark suite and methodology for stress-testing static, dynamic, symbolic, and ML-based analyzers. Extending our prior detection and evaluation work [2, 3, 4, 5, 6, 7], it aligns with recent systematic reviews [8, 9] to deliver a public leaderboard, mutation harness, and taxonomy of blind spots. Analyzer developers and auditors are invited to expand the benchmark.

Problem Formalization

Existing benchmarks are limited to static or non-adversarial test suites that do not quantify robustness against semantic-preserving obfuscations or composed multi-contract scenarios, resulting in over-optimistic tool rankings and undetected real-world blind spots. This project solves these by introducing parameterized mutation operators, differential exploit-generation trials, and a calibrated reliability score that explicitly measures error growth under increasing obfuscation intensity.

There is no comprehensive adversarial benchmark that measures where automated analyzers (static, dynamic, and ML-based) fail in realistic, obfuscated, or composed scenarios. Building such a benchmark enables trustworthy comparison and drives robustness improvements. This project creates an adversarial evaluation suite and an associated methodology for systematically stress-testing analyzers.

We define the analyzer evaluation problem by modeling an analyzer $\mathcal{A}$ as a function mapping a contract C (or a sequence of contracts) to a set of labeled issues $\mathcal{A}(C) = \{(l_i, s_i)\}$, where l_i is a label (vulnerability class) and s_i is a confidence score. For any ground-truth labeled contract set $\mathcal{D}_{\text{GT}}$, we measure analyzer performance by classical metrics (precision, recall) and additional mismatch measures that capture semantic misses (e.g., analyzer flags an issue but misses the exploit chain). To uncover blind spots, we generate adversarial variants $\tilde{C} = \mathcal{M}(C, \alpha)$ via mutation operators $\mathcal{M}$ parameterized by α controlling obfuscation intensity. Examples of operators include control-flow flattening, storage layout reordering, inline assembly injection, and API-level indirection via proxy patterns.

The core research problem is to quantify how the analyzer error rate $E(\mathcal{A}, \alpha)$ grows as a function of α and to categorize failure modes. A robust analyzer exhibits bounded growth: $\partial E / \partial \alpha$ is small. To operationalize this, we develop a taxonomy of transformations and semantic-preserving obfuscations, and we instrument mutation coverage metrics that estimate the search difficulty for each analyzer.

We also propose to evaluate analyzers in a differential setting: for a pair $(\mathcal{A}_1, \mathcal{A}_2)$, we compute regions of disagreement and perform exploit-generation trials to determine which analyzer better predicts actionable vulnerabilities. We quantify analyzer reliability using a calibrated score that combines detection performance with adversarial resilience:

$$\text{Reliability}(\mathcal{A}) = \mathbb{E}_{C \sim \mathcal{D}} \left[\text{Accuracy}(\mathcal{A}, C) \cdot \exp \left(- \beta \cdot \text{VulnGap}(\mathcal{A}, C) \right) \right],$$

where VulnGap measures the gap between analyzer output and ground-truth exploitability as evidenced by exploit traces.

Deliverables include (1) the adversarial benchmark dataset, (2) an automated mutation-and-test harness, (3) a public leaderboard and evaluation framework, and (4) an empirical taxonomy paper describing failure modes and recommendations. We explicitly include previously-published detection datasets and methods from our team as seed examples and baselines [2, 3, 4, 5]. Collaboration opportunities include partnerships with analyzer authors (Slither, Mythril, Manticore, symbolic/ML tool builders) for white-box evaluation and industry sponsors for red-team competitions. We invite tool maintainers, auditors, and academic groups to join the benchmark effort to improve the practical robustness of contract security tooling.

References

- [1] T. Abdelaziz, A. Sedky Adly, B. Rossi, and M.-S. Mostafa. Identification and assessment of software design pattern violations. *Informatics Bulletin*, 1(2):6–13, 2019. https://fcihib.journals.ekb.eg/article_107517_62d89752f7d871844b0e5dd1601da4f5.pdf.
- [2] T. Abdelaziz and A. Hobor. Smart learning to find dumb contracts. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 1775–1792, 2023. <https://www.usenix.org/conference/usenixsecurity23/presentation/abdelaziz>.
- [3] T. Abdelaziz and A. Hobor. Smart learning to find dumb contracts (extended version). arXiv:2304.10726, 2023. <https://arxiv.org/abs/2304.10726>.
- [4] T. Abdelaziz and A. Hobor. USENIX’23 artifact appendix: Smart learning to find dumb contracts. USENIX Association, 2023. <https://www.usenix.org/system/files/usenixsecurity23-appendix-abdelaziz.pdf>.
- [5] T. Abdelaziz and A. Hobor. Schooling to exploit foolish contracts. In *2023 Fifth International Conference on Blockchain Computing and Applications (BCCA)*, pages 388–395, 2023. <https://ieeexplore.ieee.org/document/10338924>.
- [6] T. A. Abdelmegid Mohamed. Towards secure smart contracts: A deep learning approach for detecting security threats. PhD thesis, National University of Singapore, 2023. <https://scholarbank.nus.edu.sg/handle/10635/247301>.
- [7] T. Abdelaziz, S. Alsaghir, and K. Ali. Where do smart contract security analyzers fall short? *Proceedings of the 2026 IEEE/ACM 23rd International Conference on Mining Software Repositories (MSR)*, 2026.
- [8] G. Iuliano et al. Smart Contract Vulnerabilities, Tools, and Benchmarks: an Updated Systematic Literature Review. arXiv:2412.01719, 2024. <https://arxiv.org/abs/2412.01719>.
- [9] Z. Li et al. SCALM: Detecting Bad Practices in Smart Contracts Through LLMs. arXiv:2502.04347, 2026. <https://arxiv.org/abs/2502.04347>.
- [10] G. Wood. Ethereum: A secure decentralised generalised transaction ledger. *Ethereum Project Yellow Paper*, 2014. <https://ethereum.github.io/yellowpaper/paper.pdf>.
- [11] L. Luu, D.-H. Chu, H. Olickel, P. Saxena, and A. Hobor. Making smart contracts smarter. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016.
- [12] N. Atzei, M. Bartoletti, and T. Cimoli. A survey of attacks on ethereum smart contracts (soK). In *Proceedings of the 6th International Conference on Principles of Security and Trust (POST)*, 2017.